Part 4 - Assess Query Performance

PostgreSQL Indexing Benchmark: Queries 1-8

Method

The benchmark ran each of the eight PostgreSQL queries once before and once after creating six proposed indexes and refreshing statistics with ANALYZE. The same query inputs were used. Values below are PostgreSQL EXPLAIN ANALYZE Execution Time, not the separate psql wall-clock Time values. The transaction ended with ROLLBACK, so the six experimental indexes were not committed. Because this is one run per condition, the percentages describe this run rather than a repeatable performance guarantee.

Performance Summary

Query	Description	Before	After	Change
Q1	Specific track in album	177.998 ms	0.366 ms	99.8% faster
Q2	Exact album	23.377 ms	0.108 ms	99.5% faster
Q3	Popular tracks in genre	6.823 ms	0.686 ms	89.9% faster
Q4	Highly rated albums	10.898 ms	1.899 ms	82.6% faster
Q5	Genre characteristics	75.913 ms	76.291 ms	0.5% slower
Q6	Artist review analysis	169.548 ms	146.032 ms	13.9% faster
Q7	Specific artist	2.001 ms	0.116 ms	94.2% faster
Q8	Albums by release period	14.878 ms	8.327 ms	44.0% faster

Results

Point queries: Q1, Q2, and Q7 had the largest percentage reductions. Q1 fell from 177.998 to 0.366 ms; the new idx_tracks_album_name located the track, while primary-key indexes also replaced broad join scans. Q2 fell from 23.377 to 0.108 ms and used idx_album_name_date. Q7 fell from 2.001 to 0.116 ms and used idx_artists_name_ctry. The baseline Q1 and Q7 scans rejected 89,740 track rows and 39,527 artist rows, respectively.

Range queries: Q3 fell from 6.823 to 0.686 ms and used a bitmap scan on idx_tracks_genre_pop. Q4 fell from 10.898 to 1.899 ms and used bitmap scans on both idx_album_genre_date and bench_album_scores_critic_idx. The latter index is outside the six proposed indexes; the log does not show when it was created, so Q4's gain cannot be attributed solely to the proposed album index. Q8 fell from 14.878 to 8.327 ms and used a bitmap scan on idx_album_release_date. Q3, Q4, and Q8 still had explicit Sort nodes; bitmap access did not preserve the requested ordering.

Analytical queries: Q5 changed from 75.913 to 76.291 ms (0.5% slower). Q6 changed from 169.548 to 146.032 ms (13.9% faster). Both plans still read large tables with sequential scans and aggregate many rows. No new index was proposed for either query; one measurement per condition cannot establish that indexing caused these small or moderate changes.

Conclusion

In this single benchmark, the proposed indexes were used for the point and range queries, and those queries ran faster. The broad analytical queries continued to scan and aggregate large portions of the data. Q4 also used an additional critic-score index, which limits attribution. Repeating each condition several times and reporting median execution time would make the comparison more reliable.

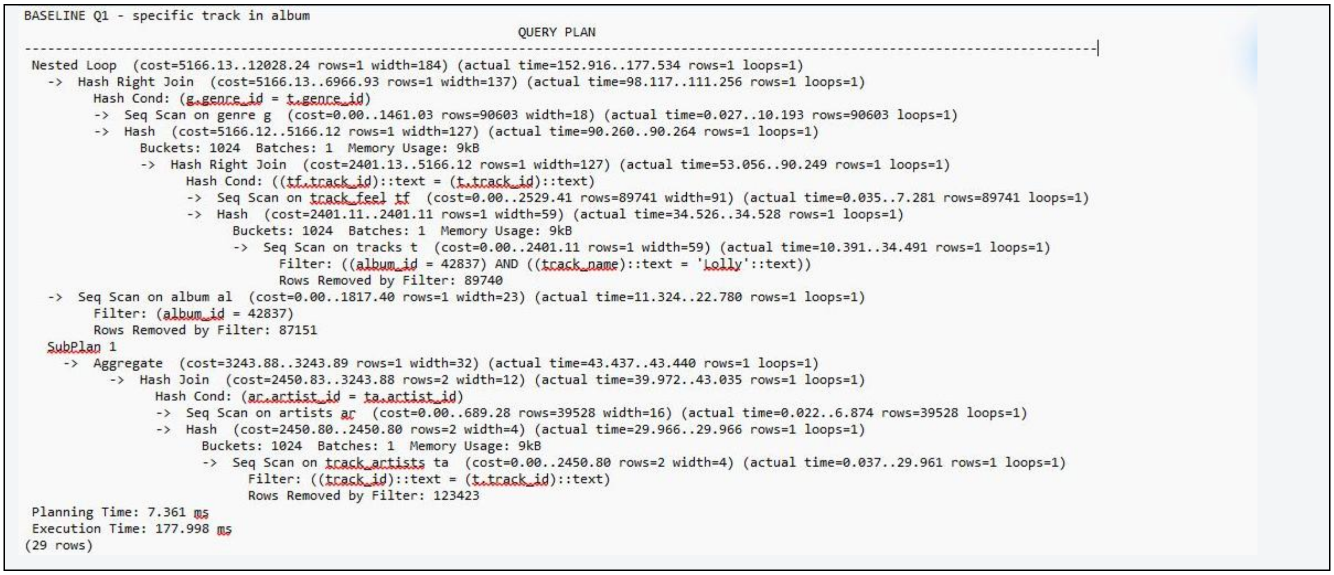
Source: index_benchmark_output.txt

Database screenshots

Q1 Specific track in album

Q1 Specific track in album — Before indexing — Execution Time: 177.998 ms

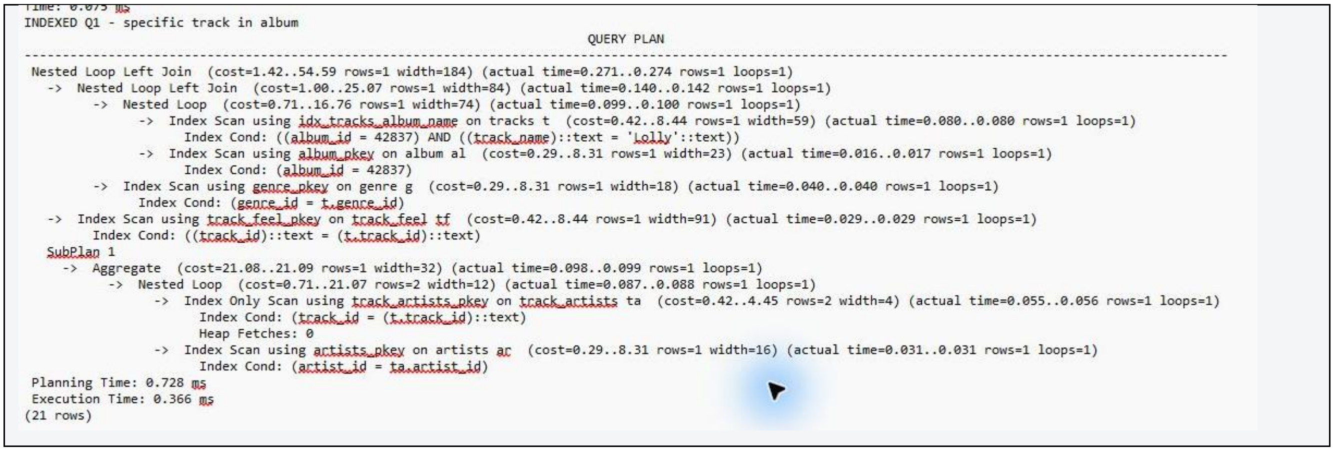
Baseline execution plan, before the six proposed indexes were created.



```
BASELINE Q1 - specific track in album
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------|
 Nested Loop  (cost=5166.13..12028.24 rows=1 width=184) (actual time=152.916..177.534 rows=1 loops=1)
   ->  Hash Right Join  (cost=5166.13..6966.93 rows=1 width=137) (actual time=98.117..111.256 rows=1 loops=1)
         Hash Cond: (g.genre_id = t.genre_id)
         ->  Seq Scan on genre g  (cost=0.00..1461.03 rows=90603 width=18) (actual time=0.027..10.193 rows=90603 loops=1)
         ->  Hash  (cost=5166.12..5166.12 rows=1 width=127) (actual time=90.260..90.264 rows=1 loops=1)
               Buckets: 1024  Batches: 1  Memory Usage: 9kB
               ->  Hash Right Join  (cost=2401.13..5166.12 rows=1 width=127) (actual time=53.056..90.249 rows=1 loops=1)
                     Hash Cond: ((tf.track_id)::text = (t.track_id)::text)
                     ->  Seq Scan on track_feel tf  (cost=0.00..2529.41 rows=89741 width=91) (actual time=0.035..7.281 rows=89741 loops=1)
                     ->  Hash  (cost=2401.11..2401.11 rows=1 width=59) (actual time=34.526..34.528 rows=1 loops=1)
                           Buckets: 1024  Batches: 1  Memory Usage: 9kB
                           ->  Seq Scan on tracks t  (cost=0.00..2401.11 rows=1 width=59) (actual time=10.391..34.491 rows=1 loops=1)
                                 Filter: ((album_id = 42837) AND ((track_name)::text = 'Lolly'::text))
                                 Rows Removed by Filter: 89740
   ->  Seq Scan on album al  (cost=0.00..1817.40 rows=1 width=23) (actual time=11.324..22.780 rows=1 loops=1)
         Filter: (album_id = 42837)
         Rows Removed by Filter: 87151
   SubPlan 1
     ->  Aggregate  (cost=3243.88..3243.89 rows=1 width=32) (actual time=43.437..43.440 rows=1 loops=1)
           ->  Hash Join  (cost=2450.83..3243.88 rows=2 width=12) (actual time=39.972..43.035 rows=1 loops=1)
                 Hash Cond: (ar.artist_id = ta.artist_id)
                 ->  Seq Scan on artists ar  (cost=0.00..689.28 rows=39528 width=16) (actual time=0.022..6.874 rows=39528 loops=1)
                 ->  Hash  (cost=2450.80..2450.80 rows=2 width=4) (actual time=29.966..29.966 rows=1 loops=1)
                       Buckets: 1024  Batches: 1  Memory Usage: 9kB
                       ->  Seq Scan on track_artists ta  (cost=0.00..2450.80 rows=2 width=4) (actual time=0.037..29.961 rows=1 loops=1)
                             Filter: ((track_id)::text = (t.track_id)::text)
                             Rows Removed by Filter: 123423
 Planning Time: 7.361 ms
 Execution Time: 177.998 ms
(29 rows)
```

Q1 Specific track in album — After indexing — Execution Time: 0.366 ms

Execution plan after the six proposed indexes were created and ANALYZE ran. Plan evidence to show: idx_tracks_album_name.

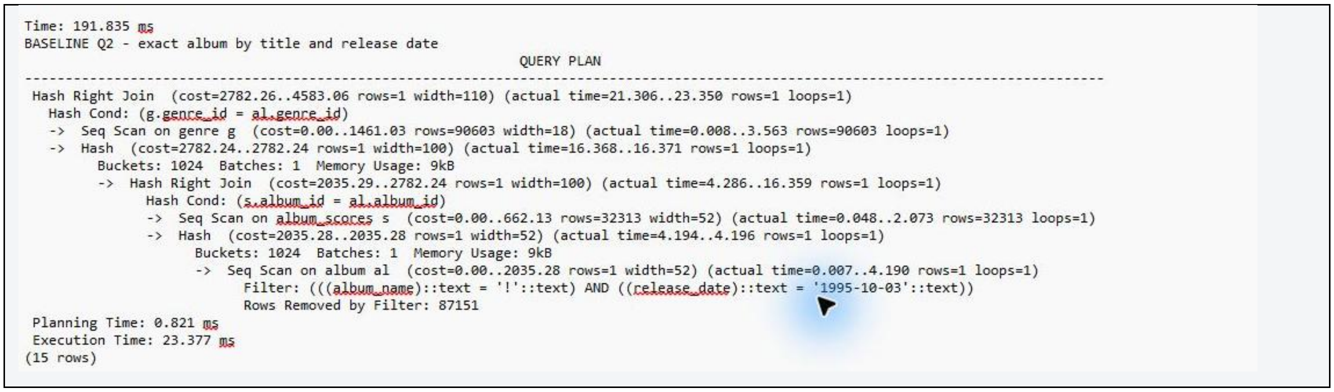


```
Time: 0.075 ms
INDEXED Q1 - specific track in album
                                                                      QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------------
 Nested Loop Left Join  (cost=1.42..54.59 rows=1 width=184) (actual time=0.271..0.274 rows=1 loops=1)
   ->  Nested Loop Left Join  (cost=1.00..25.07 rows=1 width=84) (actual time=0.140..0.142 rows=1 loops=1)
         ->  Nested Loop  (cost=0.71..16.76 rows=1 width=74) (actual time=0.099..0.100 rows=1 loops=1)
               ->  Index Scan using idx_tracks_album_name on tracks t  (cost=0.42..8.44 rows=1 width=59) (actual time=0.080..0.080 rows=1 loops=1)
                     Index Cond: ((album_id = 42837) AND ((track_name)::text = 'Lolly'::text))
               ->  Index Scan using album_pkey on album al  (cost=0.29..8.31 rows=1 width=23) (actual time=0.016..0.017 rows=1 loops=1)
                     Index Cond: (album_id = 42837)
         ->  Index Scan using genre_pkey on genre g  (cost=0.29..8.31 rows=1 width=18) (actual time=0.040..0.040 rows=1 loops=1)
               Index Cond: (genre_id = t.genre_id)
   ->  Index Scan using track_feel_pkey on track_feel tf  (cost=0.42..8.44 rows=1 width=91) (actual time=0.029..0.029 rows=1 loops=1)
         Index Cond: ((track_id)::text = (t.track_id)::text)
   SubPlan 1
     ->  Aggregate  (cost=21.08..21.09 rows=1 width=32) (actual time=0.098..0.099 rows=1 loops=1)
           ->  Nested Loop  (cost=0.71..21.07 rows=2 width=12) (actual time=0.087..0.088 rows=1 loops=1)
                 ->  Index Only Scan using track_artists_pkey on track_artists ta  (cost=0.42..4.45 rows=2 width=4) (actual time=0.055..0.056 rows=1 loops=1)
                       Index Cond: (track_id = (t.track_id)::text)
                       Heap Fetches: 0
                 ->  Index Scan using artists_pkey on artists ar  (cost=0.29..8.31 rows=1 width=16) (actual time=0.031..0.031 rows=1 loops=1)
                       Index Cond: (artist_id = ta.artist_id)
 Planning Time: 0.728 ms
 Execution Time: 0.366 ms
(21 rows)
```

Q2 Exact album by title and date

Q2 Exact album by title and date — Before indexing — Execution Time: 23.377 ms

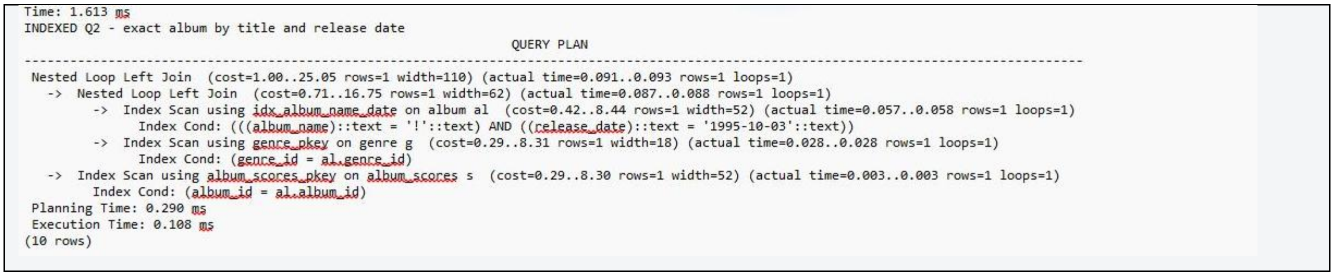
Baseline execution plan, before the six proposed indexes were created.



```
Time: 191.835 ms
BASELINE Q2 - exact album by title and release date
                                                            QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------
 Hash Right Join  (cost=2782.26..4583.06 rows=1 width=110) (actual time=21.306..23.350 rows=1 loops=1)
   Hash Cond: (g.genre_id = al.genre_id)
   ->  Seq Scan on genre g  (cost=0.00..1461.03 rows=90603 width=18) (actual time=0.008..3.563 rows=90603 loops=1)
   ->  Hash  (cost=2782.24..2782.24 rows=1 width=100) (actual time=16.368..16.371 rows=1 loops=1)
         Buckets: 1024  Batches: 1  Memory Usage: 9kB
         ->  Hash Right Join  (cost=2035.29..2782.24 rows=1 width=100) (actual time=4.286..16.359 rows=1 loops=1)
               Hash Cond: (s.album_id = al.album_id)
               ->  Seq Scan on album_scores s  (cost=0.00..662.13 rows=32313 width=52) (actual time=0.048..2.073 rows=32313 loops=1)
               ->  Hash  (cost=2035.28..2035.28 rows=1 width=52) (actual time=4.194..4.196 rows=1 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     ->  Seq Scan on album al  (cost=0.00..2035.28 rows=1 width=52) (actual time=0.007..4.190 rows=1 loops=1)
                           Filter: (((album_name)::text = '!'::text) AND ((release_date)::text = '1995-10-03'::text))
                           Rows Removed by Filter: 87151
 Planning Time: 0.821 ms
 Execution Time: 23.377 ms
(15 rows)
```

Q2 Exact album by title and date — After indexing — Execution Time: 0.108 ms

Execution plan after the six proposed indexes were created and ANALYZE ran. Plan evidence to show: idx_album_name_date.

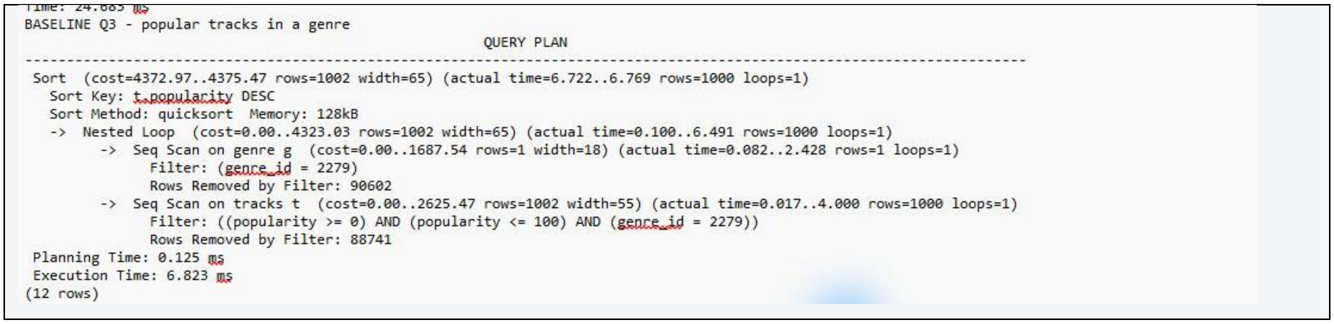


```
Time: 1.613 ms
INDEXED Q2 - exact album by title and release date
                                                            QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------
 Nested Loop Left Join  (cost=1.00..25.05 rows=1 width=110) (actual time=0.091..0.093 rows=1 loops=1)
   ->  Nested Loop Left Join  (cost=0.71..16.75 rows=1 width=62) (actual time=0.087..0.088 rows=1 loops=1)
         ->  Index Scan using idx_album_name_date on album al  (cost=0.42..8.44 rows=1 width=52) (actual time=0.057..0.058 rows=1 loops=1)
               Index Cond: (((album_name)::text = '!'::text) AND ((release_date)::text = '1995-10-03'::text))
         ->  Index Scan using genre_pkey on genre g  (cost=0.29..8.31 rows=1 width=18) (actual time=0.028..0.028 rows=1 loops=1)
               Index Cond: (genre_id = al.genre_id)
   ->  Index Scan using album_scores_pkey on album_scores s  (cost=0.29..8.30 rows=1 width=52) (actual time=0.003..0.003 rows=1 loops=1)
         Index Cond: (album_id = al.album_id)
 Planning Time: 0.290 ms
 Execution Time: 0.108 ms
(10 rows)
```

Q3 Popular tracks in a genre

Q3 Popular tracks in a genre — Before indexing — Execution Time: 6.823 ms

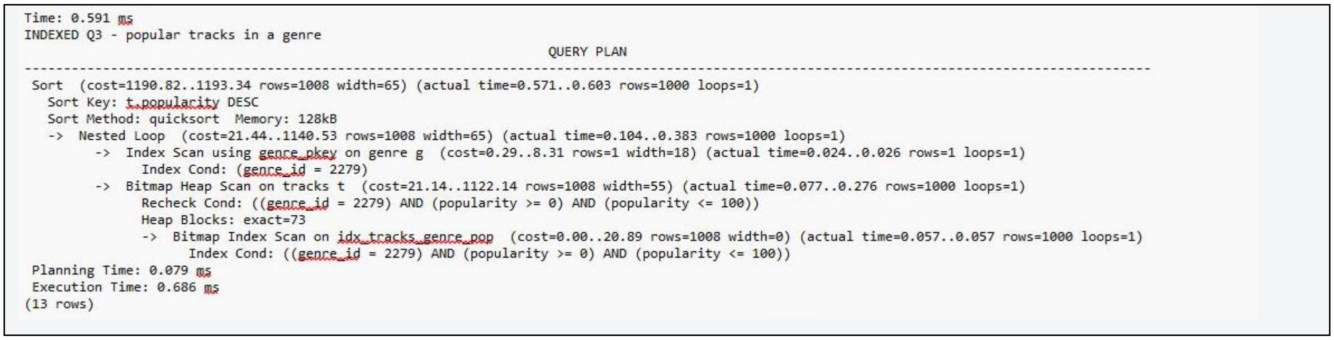
Baseline execution plan, before the six proposed indexes were created.



```
Time: 24.683 ms
BASELINE Q3 - popular tracks in a genre
                                                      QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=4372.97..4375.47 rows=1002 width=65) (actual time=6.722..6.769 rows=1000 loops=1)
   Sort Key: t.popularity DESC
   Sort Method: quicksort  Memory: 128kB
   ->  Nested Loop  (cost=0.00..4323.03 rows=1002 width=65) (actual time=0.100..6.491 rows=1000 loops=1)
         ->  Seq Scan on genre g  (cost=0.00..1687.54 rows=1 width=18) (actual time=0.082..2.428 rows=1 loops=1)
               Filter: (genre_id = 2279)
               Rows Removed by Filter: 90602
         ->  Seq Scan on tracks t  (cost=0.00..2625.47 rows=1002 width=55) (actual time=0.017..4.000 rows=1000 loops=1)
               Filter: ((popularity >= 0) AND (popularity <= 100) AND (genre_id = 2279))
               Rows Removed by Filter: 88741
 Planning Time: 0.125 ms
 Execution Time: 6.823 ms
(12 rows)
```

Q3 Popular tracks in a genre — After indexing — Execution Time: 0.686 ms

Execution plan after the six proposed indexes were created and ANALYZE ran. Plan evidence to show: idx_tracks_genre_pop.

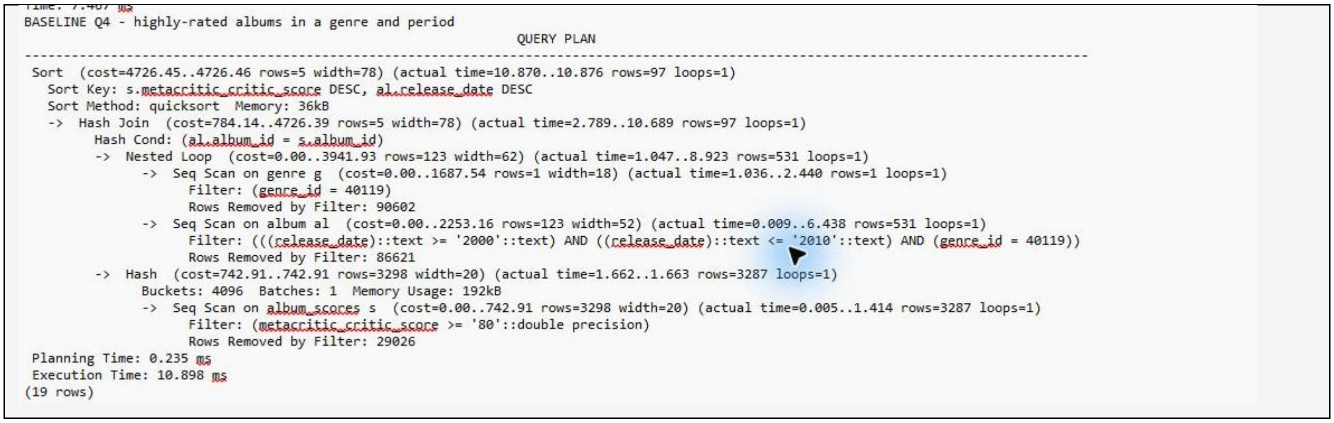


```
Time: 0.591 ms
INDEXED Q3 - popular tracks in a genre
                                                              QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=1190.82..1193.34 rows=1008 width=65) (actual time=0.571..0.603 rows=1000 loops=1)
   Sort Key: t.popularity DESC
   Sort Method: quicksort  Memory: 128kB
   ->  Nested Loop  (cost=21.44..1140.53 rows=1008 width=65) (actual time=0.104..0.383 rows=1000 loops=1)
         ->  Index Scan using genre_pkey on genre g  (cost=0.29..8.31 rows=1 width=18) (actual time=0.024..0.026 rows=1 loops=1)
               Index Cond: (genre_id = 2279)
         ->  Bitmap Heap Scan on tracks t  (cost=21.14..1122.14 rows=1008 width=55) (actual time=0.077..0.276 rows=1000 loops=1)
               Recheck Cond: ((genre_id = 2279) AND (popularity >= 0) AND (popularity <= 100))
               Heap Blocks: exact=73
               ->  Bitmap Index Scan on idx_tracks_genre_pop  (cost=0.00..20.89 rows=1008 width=0) (actual time=0.057..0.057 rows=1000 loops=1)
                     Index Cond: ((genre_id = 2279) AND (popularity >= 0) AND (popularity <= 100))
 Planning Time: 0.079 ms
 Execution Time: 0.686 ms
(13 rows)
```

Q4 Highly rated albums in a genre and period

Q4 Highly rated albums in a genre and period — Before indexing — Execution Time: 10.898 ms

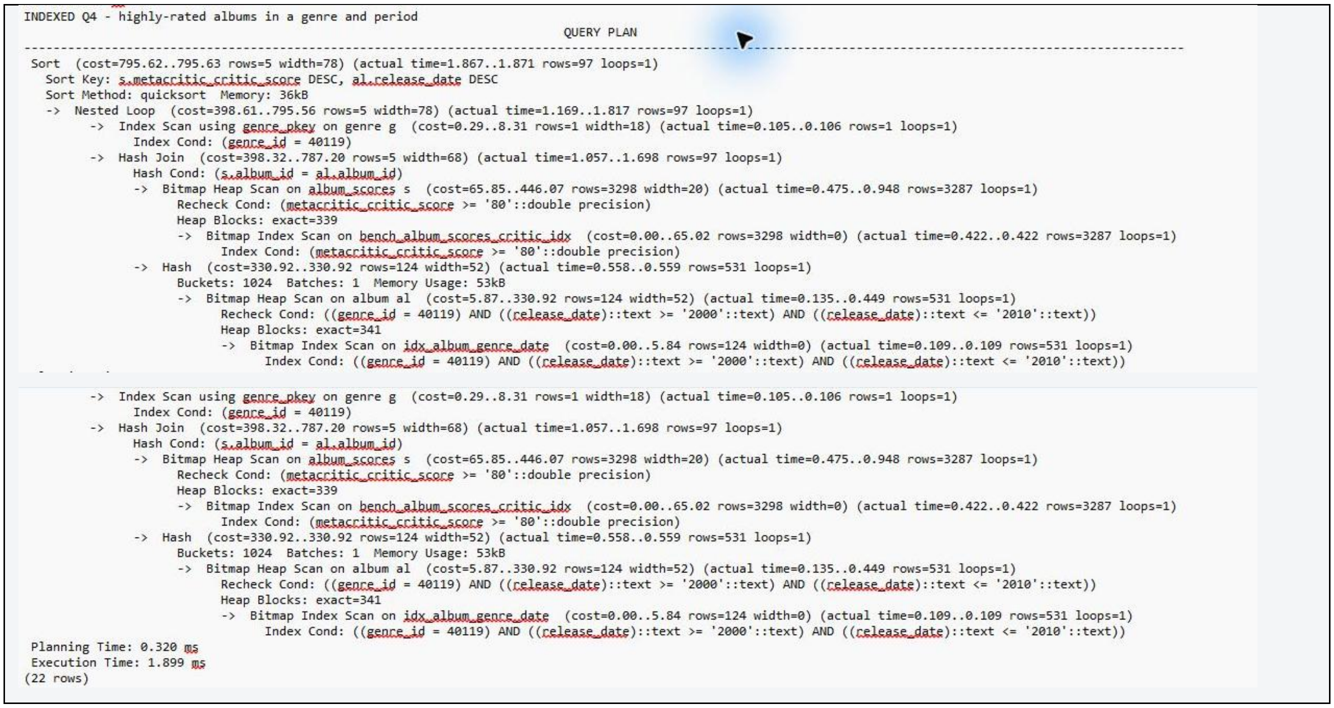
Baseline execution plan, before the six proposed indexes were created.



```
BASELINE Q4 - highly-rated albums in a genre and period
                                                                          QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=4726.45..4726.46 rows=5 width=78) (actual time=10.870..10.876 rows=97 loops=1)
   Sort Key: s.metacritic_critic_score DESC, al.release_date DESC
   Sort Method: quicksort  Memory: 36kB
   ->  Hash Join  (cost=784.14..4726.39 rows=5 width=78) (actual time=2.789..10.689 rows=97 loops=1)
         Hash Cond: (al.album_id = s.album_id)
         ->  Nested Loop  (cost=0.00..3941.93 rows=123 width=62) (actual time=1.047..8.923 rows=531 loops=1)
               ->  Seq Scan on genre g  (cost=0.00..1687.54 rows=1 width=18) (actual time=1.036..2.440 rows=1 loops=1)
                     Filter: (genre_id = 40119)
                     Rows Removed by Filter: 90602
               ->  Seq Scan on album al  (cost=0.00..2253.16 rows=123 width=52) (actual time=0.009..6.438 rows=531 loops=1)
                     Filter: (((release_date)::text >= '2000'::text) AND ((release_date)::text <= '2010'::text) AND (genre_id = 40119))
                     Rows Removed by Filter: 86621
         ->  Hash  (cost=742.91..742.91 rows=3298 width=20) (actual time=1.662..1.663 rows=3287 loops=1)
               Buckets: 4096  Batches: 1  Memory Usage: 192kB
               ->  Seq Scan on album_scores s  (cost=0.00..742.91 rows=3298 width=20) (actual time=0.005..1.414 rows=3287 loops=1)
                     Filter: (metacritic_critic_score >= '80'::double precision)
                     Rows Removed by Filter: 29026
 Planning Time: 0.235 ms
 Execution Time: 10.898 ms
(19 rows)
```

Q4 Highly rated albums in a genre and period — After indexing — Execution Time: 1.899 ms

Execution plan after the six proposed indexes were created and ANALYZE ran. Plan evidence to show: idx_album_genre_date and bench_album_scores_critic_idx.



```
INDEXED Q4 - highly-rated albums in a genre and period
                                                                          QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=795.62..795.63 rows=5 width=78) (actual time=1.867..1.871 rows=97 loops=1)
   Sort Key: s.metacritic_critic_score DESC, al.release_date DESC
   Sort Method: quicksort  Memory: 36kB
   ->  Nested Loop  (cost=398.61..795.56 rows=5 width=78) (actual time=1.169..1.817 rows=97 loops=1)
         ->  Index Scan using genre_pkey on genre g  (cost=0.29..8.31 rows=1 width=18) (actual time=0.105..0.106 rows=1 loops=1)
               Index Cond: (genre_id = 40119)
         ->  Hash Join  (cost=398.32..787.20 rows=5 width=68) (actual time=1.057..1.698 rows=97 loops=1)
               Hash Cond: (s.album_id = al.album_id)
               ->  Bitmap Heap Scan on album_scores s  (cost=65.85..446.07 rows=3298 width=20) (actual time=0.475..0.948 rows=3287 loops=1)
                     Recheck Cond: (metacritic_critic_score >= '80'::double precision)
                     Heap Blocks: exact=339
                     ->  Bitmap Index Scan on bench_album_scores_critic_idx  (cost=0.00..65.02 rows=3298 width=0) (actual time=0.422..0.422 rows=3287 loops=1)
                           Index Cond: (metacritic_critic_score >= '80'::double precision)
               ->  Hash  (cost=330.92..330.92 rows=124 width=52) (actual time=0.558..0.559 rows=531 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 53kB
                     ->  Bitmap Heap Scan on album al  (cost=5.87..330.92 rows=124 width=52) (actual time=0.135..0.449 rows=531 loops=1)
                           Recheck Cond: ((genre_id = 40119) AND ((release_date)::text >= '2000'::text) AND ((release_date)::text <= '2010'::text))
                           Heap Blocks: exact=341
                           ->  Bitmap Index Scan on idx_album_genre_date  (cost=0.00..5.84 rows=124 width=0) (actual time=0.109..0.109 rows=531 loops=1)
                                 Index Cond: ((genre_id = 40119) AND ((release_date)::text >= '2000'::text) AND ((release_date)::text <= '2010'::text))
```

```
         ->  Index Scan using genre_pkey on genre g  (cost=0.29..8.31 rows=1 width=18) (actual time=0.105..0.106 rows=1 loops=1)
               Index Cond: (genre_id = 40119)
         ->  Hash Join  (cost=398.32..787.20 rows=5 width=68) (actual time=1.057..1.698 rows=97 loops=1)
               Hash Cond: (s.album_id = al.album_id)
               ->  Bitmap Heap Scan on album_scores s  (cost=65.85..446.07 rows=3298 width=20) (actual time=0.475..0.948 rows=3287 loops=1)
                     Recheck Cond: (metacritic_critic_score >= '80'::double precision)
                     Heap Blocks: exact=339
                     ->  Bitmap Index Scan on bench_album_scores_critic_idx  (cost=0.00..65.02 rows=3298 width=0) (actual time=0.422..0.422 rows=3287 loops=1)
                           Index Cond: (metacritic_critic_score >= '80'::double precision)
               ->  Hash  (cost=330.92..330.92 rows=124 width=52) (actual time=0.558..0.559 rows=531 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 53kB
                     ->  Bitmap Heap Scan on album al  (cost=5.87..330.92 rows=124 width=52) (actual time=0.135..0.449 rows=531 loops=1)
                           Recheck Cond: ((genre_id = 40119) AND ((release_date)::text >= '2000'::text) AND ((release_date)::text <= '2010'::text))
                           Heap Blocks: exact=341
                           ->  Bitmap Index Scan on idx_album_genre_date  (cost=0.00..5.84 rows=124 width=0) (actual time=0.109..0.109 rows=531 loops=1)
                                 Index Cond: ((genre_id = 40119) AND ((release_date)::text >= '2000'::text) AND ((release_date)::text <= '2010'::text))
```

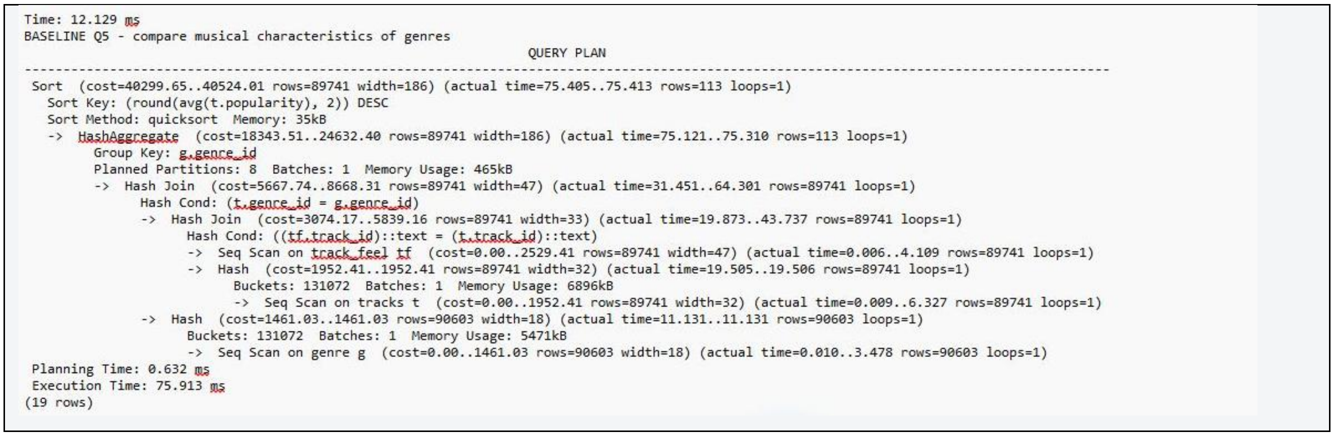
```
 Planning Time: 0.320 ms
 Execution Time: 1.899 ms
(22 rows)
```

Q4 note: The after plan also uses bench_album_scores_critic_idx, which is outside the six proposed indexes. Keep it visible in the screenshot so the report does not overattribute the improvement.

Q5 Genre characteristics

Q5 Genre characteristics — Before indexing — Execution Time: 75.913 ms

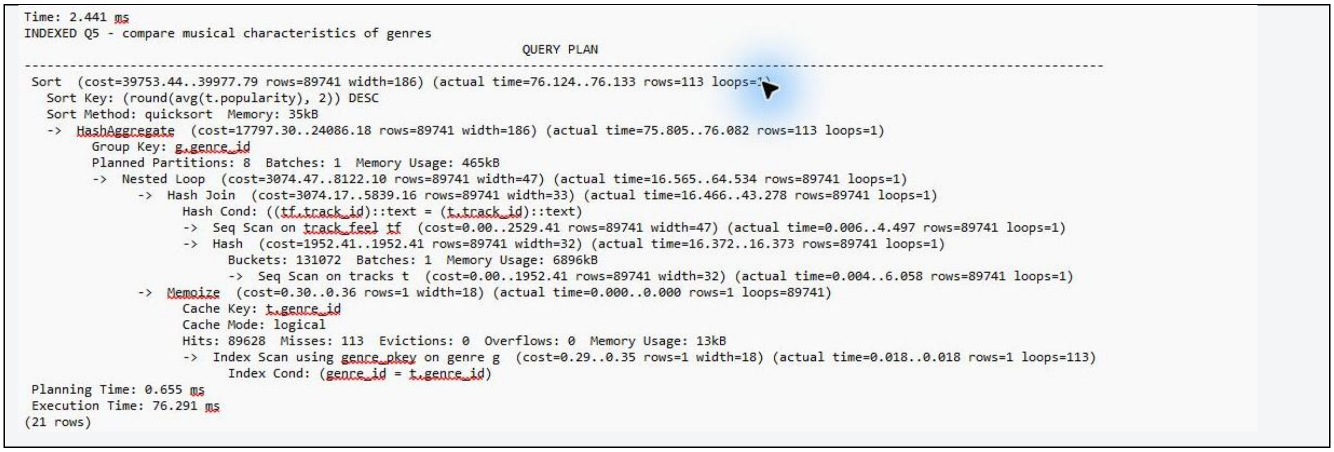
Baseline execution plan, before the six proposed indexes were created.



```
Time: 12.129 ms
BASELINE Q5 - compare musical characteristics of genres
                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=40299.65..40524.01 rows=89741 width=186) (actual time=75.405..75.413 rows=113 loops=1)
   Sort Key: (round(avg(t.popularity), 2)) DESC
   Sort Method: quicksort  Memory: 35kB
   ->  HashAggregate  (cost=18343.51..24632.40 rows=89741 width=186) (actual time=75.121..75.310 rows=113 loops=1)
         Group Key: g.genre_id
         Planned Partitions: 8  Batches: 1  Memory Usage: 465kB
         ->  Hash Join  (cost=5667.74..8668.31 rows=89741 width=47) (actual time=31.451..64.301 rows=89741 loops=1)
               Hash Cond: (t.genre_id = g.genre_id)
               ->  Hash Join  (cost=3074.17..5839.16 rows=89741 width=33) (actual time=19.873..43.737 rows=89741 loops=1)
                     Hash Cond: ((tf.track_id)::text = (t.track_id)::text)
                     ->  Seq Scan on track_feel tf  (cost=0.00..2529.41 rows=89741 width=47) (actual time=0.006..4.109 rows=89741 loops=1)
                     ->  Hash  (cost=1952.41..1952.41 rows=89741 width=32) (actual time=19.505..19.506 rows=89741 loops=1)
                           Buckets: 131072  Batches: 1  Memory Usage: 6896kB
                           ->  Seq Scan on tracks t  (cost=0.00..1952.41 rows=89741 width=32) (actual time=0.009..6.327 rows=89741 loops=1)
               ->  Hash  (cost=1461.03..1461.03 rows=90603 width=18) (actual time=11.131..11.131 rows=90603 loops=1)
                     Buckets: 131072  Batches: 1  Memory Usage: 5471kB
                     ->  Seq Scan on genre g  (cost=0.00..1461.03 rows=90603 width=18) (actual time=0.010..3.478 rows=90603 loops=1)
 Planning Time: 0.632 ms
 Execution Time: 75.913 ms
(19 rows)
```

Q5 Genre characteristics — After indexing — Execution Time: 76.291 ms

Execution plan after the six proposed indexes were created and ANALYZE ran. Plan evidence to show: broad scans and aggregation; no new index proposed.

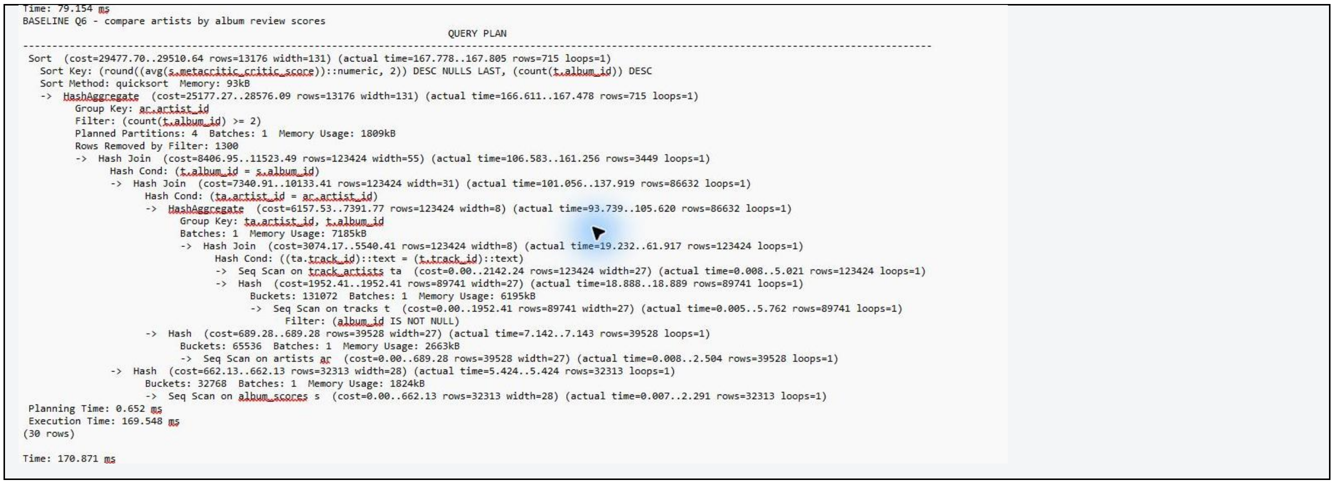


```
Time: 2.441 ms
INDEXED Q5 - compare musical characteristics of genres
                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=39753.44..39977.79 rows=89741 width=186) (actual time=76.124..76.133 rows=113 loops=1)
   Sort Key: (round(avg(t.popularity), 2)) DESC
   Sort Method: quicksort  Memory: 35kB
   ->  HashAggregate  (cost=17797.30..24086.18 rows=89741 width=186) (actual time=75.805..76.082 rows=113 loops=1)
         Group Key: g.genre_id
         Planned Partitions: 8  Batches: 1  Memory Usage: 465kB
         ->  Nested Loop  (cost=3074.47..8122.10 rows=89741 width=47) (actual time=16.565..64.534 rows=89741 loops=1)
               ->  Hash Join  (cost=3074.17..5839.16 rows=89741 width=33) (actual time=16.466..43.278 rows=89741 loops=1)
                     Hash Cond: ((tf.track_id)::text = (t.track_id)::text)
                     ->  Seq Scan on track_feel tf  (cost=0.00..2529.41 rows=89741 width=47) (actual time=0.006..4.497 rows=89741 loops=1)
                     ->  Hash  (cost=1952.41..1952.41 rows=89741 width=32) (actual time=16.372..16.373 rows=89741 loops=1)
                           Buckets: 131072  Batches: 1  Memory Usage: 6896kB
                           ->  Seq Scan on tracks t  (cost=0.00..1952.41 rows=89741 width=32) (actual time=0.004..6.058 rows=89741 loops=1)
               ->  Memoize  (cost=0.30..0.36 rows=1 width=18) (actual time=0.000..0.000 rows=1 loops=89741)
                     Cache Key: t.genre_id
                     Cache Mode: logical
                     Hits: 89628  Misses: 113  Evictions: 0  Overflows: 0  Memory Usage: 13kB
                     ->  Index Scan using genre_pkey on genre g  (cost=0.29..0.35 rows=1 width=18) (actual time=0.018..0.018 rows=1 loops=113)
                           Index Cond: (genre_id = t.genre_id)
 Planning Time: 0.655 ms
 Execution Time: 76.291 ms
(21 rows)
```

Q6 Artist review analysis

Q6 Artist review analysis — Before indexing — Execution Time: 169.548 ms

Baseline execution plan, before the six proposed indexes were created.

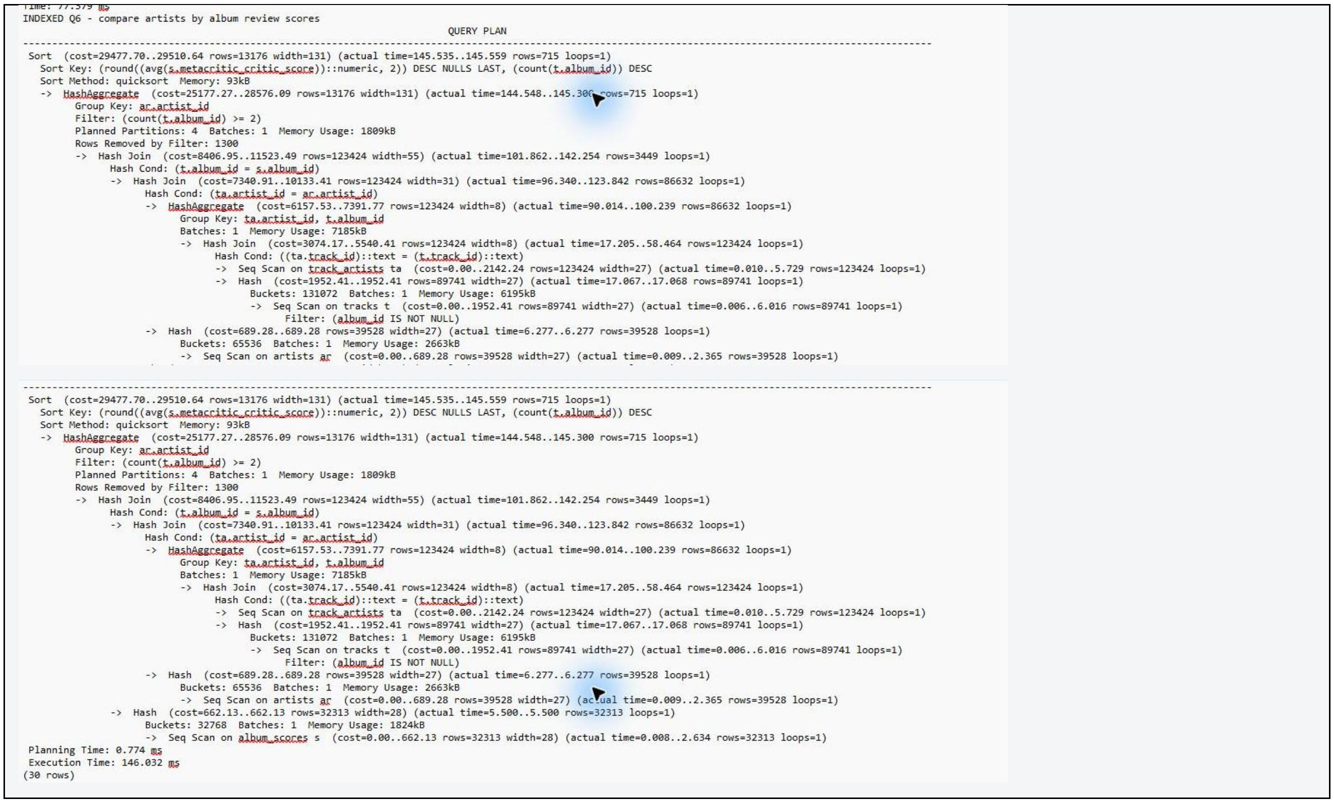


```
Time: 79.154 ms
BASELINE Q6 - compare artists by album review scores
                                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=29477.70..29510.64 rows=13176 width=131) (actual time=167.778..167.805 rows=715 loops=1)
   Sort Key: (round((avg(s.metacritic_critic_score))::numeric, 2)) DESC NULLS LAST, (count(t.album_id)) DESC
   Sort Method: quicksort  Memory: 93kB
   ->  HashAggregate  (cost=25177.27..28576.09 rows=13176 width=131) (actual time=166.611..167.478 rows=715 loops=1)
         Group Key: ar.artist_id
         Filter: (count(t.album_id) >= 2)
         Planned Partitions: 4  Batches: 1  Memory Usage: 1809kB
         Rows Removed by Filter: 1300
         ->  Hash Join  (cost=8406.95..11523.49 rows=123424 width=55) (actual time=106.583..161.256 rows=3449 loops=1)
               Hash Cond: (t.album_id = s.album_id)
               ->  Hash Join  (cost=7340.91..10133.41 rows=123424 width=31) (actual time=101.056..137.919 rows=86632 loops=1)
                     Hash Cond: (ta.artist_id = ar.artist_id)
                     ->  HashAggregate  (cost=6157.53..7391.77 rows=123424 width=8) (actual time=93.739..105.620 rows=86632 loops=1)
                           Group Key: ta.artist_id, t.album_id
                           Batches: 1  Memory Usage: 7185kB
                           ->  Hash Join  (cost=3074.17..5540.41 rows=123424 width=8) (actual time=19.232..61.917 rows=123424 loops=1)
                                 Hash Cond: ((ta.track_id)::text = (t.track_id)::text)
                                 ->  Seq Scan on track_artists ta  (cost=0.00..2142.24 rows=123424 width=27) (actual time=0.008..5.021 rows=123424 loops=1)
                                 ->  Hash  (cost=1952.41..1952.41 rows=89741 width=27) (actual time=18.888..18.889 rows=89741 loops=1)
                                       Buckets: 131072  Batches: 1  Memory Usage: 6195kB
                                       ->  Seq Scan on tracks t  (cost=0.00..1952.41 rows=89741 width=27) (actual time=0.005..5.762 rows=89741 loops=1)
                                             Filter: (album_id IS NOT NULL)
                     ->  Hash  (cost=689.28..689.28 rows=39528 width=27) (actual time=7.142..7.143 rows=39528 loops=1)
                           Buckets: 65536  Batches: 1  Memory Usage: 2663kB
                           ->  Seq Scan on artists ar  (cost=0.00..689.28 rows=39528 width=27) (actual time=0.008..2.504 rows=39528 loops=1)
               ->  Hash  (cost=662.13..662.13 rows=32313 width=28) (actual time=5.424..5.424 rows=32313 loops=1)
                     Buckets: 32768  Batches: 1  Memory Usage: 1824kB
                     ->  Seq Scan on album_scores s  (cost=0.00..662.13 rows=32313 width=28) (actual time=0.007..2.291 rows=32313 loops=1)
 Planning Time: 0.652 ms
 Execution Time: 169.548 ms
(30 rows)

Time: 170.871 ms
```

Q6 Artist review analysis — After indexing — Execution Time: 146.032 ms

Execution plan after the six proposed indexes were created and ANALYZE ran. Plan evidence to show: broad scans and aggregation; no new index proposed.



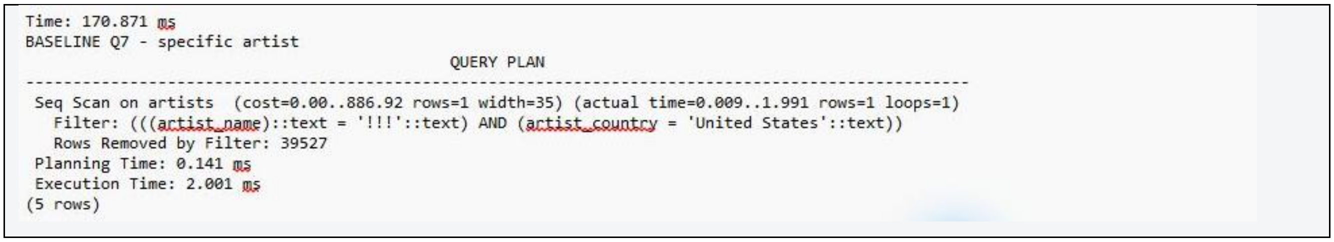
```
Time: 77.379 ms
INDEXED Q6 - compare artists by album review scores
                                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=29477.70..29510.64 rows=13176 width=131) (actual time=145.535..145.559 rows=715 loops=1)
   Sort Key: (round((avg(s.metacritic_critic_score))::numeric, 2)) DESC NULLS LAST, (count(t.album_id)) DESC
   Sort Method: quicksort  Memory: 93kB
   ->  HashAggregate  (cost=25177.27..28576.09 rows=13176 width=131) (actual time=144.548..145.300 rows=715 loops=1)
         Group Key: ar.artist_id
         Filter: (count(t.album_id) >= 2)
         Planned Partitions: 4  Batches: 1  Memory Usage: 1809kB
         Rows Removed by Filter: 1300
         ->  Hash Join  (cost=8406.95..11523.49 rows=123424 width=55) (actual time=101.862..142.254 rows=3449 loops=1)
               Hash Cond: (t.album_id = s.album_id)
               ->  Hash Join  (cost=7340.91..10133.41 rows=123424 width=31) (actual time=96.340..123.842 rows=86632 loops=1)
                     Hash Cond: (ta.artist_id = ar.artist_id)
                     ->  HashAggregate  (cost=6157.53..7391.77 rows=123424 width=8) (actual time=90.014..100.239 rows=86632 loops=1)
                           Group Key: ta.artist_id, t.album_id
                           Batches: 1  Memory Usage: 7185kB
                           ->  Hash Join  (cost=3074.17..5540.41 rows=123424 width=8) (actual time=17.205..58.464 rows=123424 loops=1)
                                 Hash Cond: ((ta.track_id)::text = (t.track_id)::text)
                                 ->  Seq Scan on track_artists ta  (cost=0.00..2142.24 rows=123424 width=27) (actual time=0.010..5.729 rows=123424 loops=1)
                                 ->  Hash  (cost=1952.41..1952.41 rows=89741 width=27) (actual time=17.067..17.068 rows=89741 loops=1)
                                       Buckets: 131072  Batches: 1  Memory Usage: 6195kB
                                       ->  Seq Scan on tracks t  (cost=0.00..1952.41 rows=89741 width=27) (actual time=0.006..6.016 rows=89741 loops=1)
                                             Filter: (album_id IS NOT NULL)
                     ->  Hash  (cost=689.28..689.28 rows=39528 width=27) (actual time=6.277..6.277 rows=39528 loops=1)
                           Buckets: 65536  Batches: 1  Memory Usage: 2663kB
                           ->  Seq Scan on artists ar  (cost=0.00..689.28 rows=39528 width=27) (actual time=0.009..2.365 rows=39528 loops=1)
```

```
------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=29477.70..29510.64 rows=13176 width=131) (actual time=145.535..145.559 rows=715 loops=1)
   Sort Key: (round((avg(s.metacritic_critic_score))::numeric, 2)) DESC NULLS LAST, (count(t.album_id)) DESC
   Sort Method: quicksort  Memory: 93kB
   ->  HashAggregate  (cost=25177.27..28576.09 rows=13176 width=131) (actual time=144.548..145.300 rows=715 loops=1)
         Group Key: ar.artist_id
         Filter: (count(t.album_id) >= 2)
         Planned Partitions: 4  Batches: 1  Memory Usage: 1809kB
         Rows Removed by Filter: 1300
         ->  Hash Join  (cost=8406.95..11523.49 rows=123424 width=55) (actual time=101.862..142.254 rows=3449 loops=1)
               Hash Cond: (t.album_id = s.album_id)
               ->  Hash Join  (cost=7340.91..10133.41 rows=123424 width=31) (actual time=96.340..123.842 rows=86632 loops=1)
                     Hash Cond: (ta.artist_id = ar.artist_id)
                     ->  HashAggregate  (cost=6157.53..7391.77 rows=123424 width=8) (actual time=90.014..100.239 rows=86632 loops=1)
                           Group Key: ta.artist_id, t.album_id
                           Batches: 1  Memory Usage: 7185kB
                           ->  Hash Join  (cost=3074.17..5540.41 rows=123424 width=8) (actual time=17.205..58.464 rows=123424 loops=1)
                                 Hash Cond: ((ta.track_id)::text = (t.track_id)::text)
                                 ->  Seq Scan on track_artists ta  (cost=0.00..2142.24 rows=123424 width=27) (actual time=0.010..5.729 rows=123424 loops=1)
                                 ->  Hash  (cost=1952.41..1952.41 rows=89741 width=27) (actual time=17.067..17.068 rows=89741 loops=1)
                                       Buckets: 131072  Batches: 1  Memory Usage: 6195kB
                                       ->  Seq Scan on tracks t  (cost=0.00..1952.41 rows=89741 width=27) (actual time=0.006..6.016 rows=89741 loops=1)
                                             Filter: (album_id IS NOT NULL)
                     ->  Hash  (cost=689.28..689.28 rows=39528 width=27) (actual time=6.277..6.277 rows=39528 loops=1)
                           Buckets: 65536  Batches: 1  Memory Usage: 2663kB
                           ->  Seq Scan on artists ar  (cost=0.00..689.28 rows=39528 width=27) (actual time=0.009..2.365 rows=39528 loops=1)
               ->  Hash  (cost=662.13..662.13 rows=32313 width=28) (actual time=5.500..5.500 rows=32313 loops=1)
                     Buckets: 32768  Batches: 1  Memory Usage: 1824kB
                     ->  Seq Scan on album_scores s  (cost=0.00..662.13 rows=32313 width=28) (actual time=0.008..2.634 rows=32313 loops=1)
 Planning Time: 0.774 ms
 Execution Time: 146.032 ms
(30 rows)
```

Q7 Specific artist

Q7 Specific artist — Before indexing — Execution Time: 2.001 ms

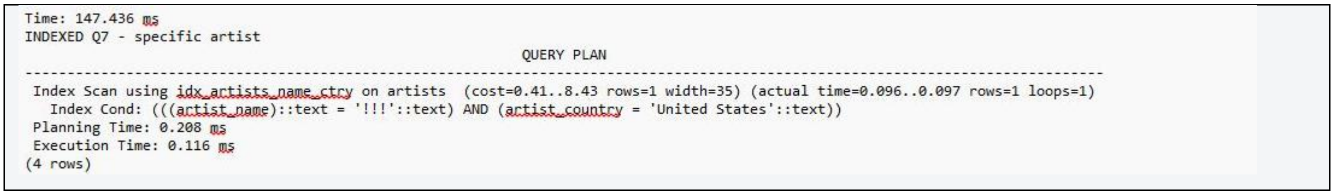
Baseline execution plan, before the six proposed indexes were created.



```
Time: 170.871 ms
BASELINE Q7 - specific artist
                                             QUERY PLAN
----------------------------------------------------------------------------------------------------
 Seq Scan on artists  (cost=0.00..886.92 rows=1 width=35) (actual time=0.009..1.991 rows=1 loops=1)
   Filter: (((artist_name)::text = '!!!'::text) AND (artist_country = 'United States'::text))
   Rows Removed by Filter: 39527
 Planning Time: 0.141 ms
 Execution Time: 2.001 ms
(5 rows)
```

Q7 Specific artist — After indexing — Execution Time: 0.116 ms

Execution plan after the six proposed indexes were created and ANALYZE ran. Plan evidence to show: idx_artists_name_ctry.

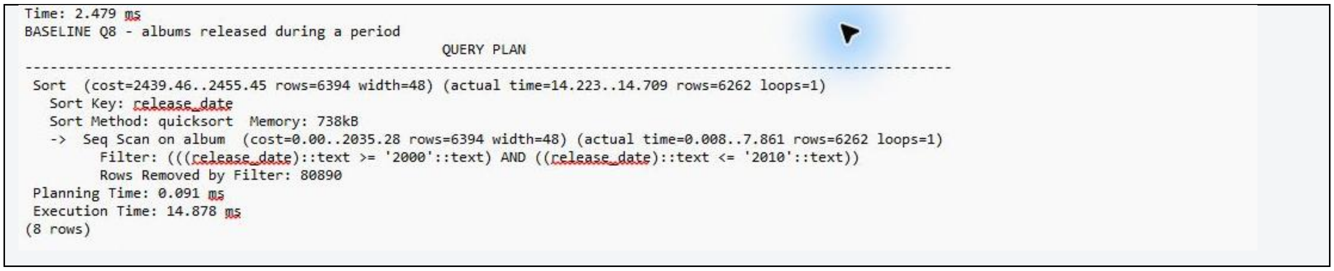


```
Time: 147.436 ms
INDEXED Q7 - specific artist
                                                          QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------
 Index Scan using idx_artists_name_ctry on artists  (cost=0.41..8.43 rows=1 width=35) (actual time=0.096..0.097 rows=1 loops=1)
   Index Cond: (((artist_name)::text = '!!!'::text) AND (artist_country = 'United States'::text))
 Planning Time: 0.208 ms
 Execution Time: 0.116 ms
(4 rows)
```

Q8 Albums by release period

Q8 Albums by release period — Before indexing — Execution Time: 14.878 ms

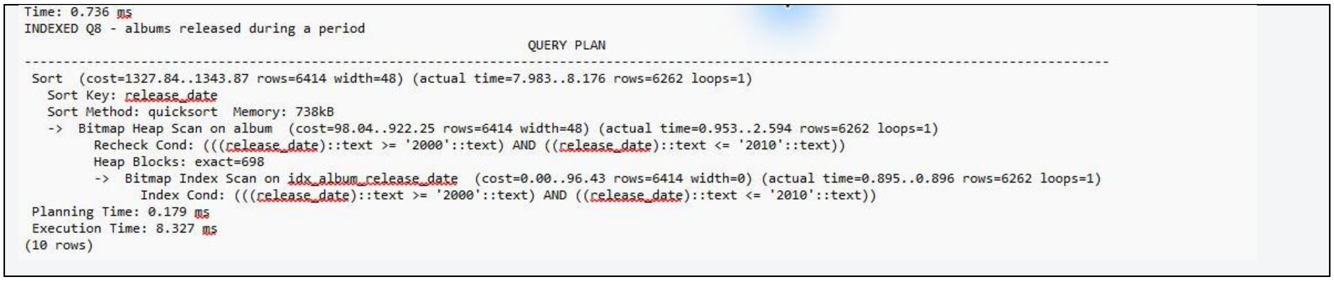
Baseline execution plan, before the six proposed indexes were created.



```
Time: 2.479 ms
BASELINE Q8 - albums released during a period
                                                  QUERY PLAN
---------------------------------------------------------------------------------------------------------------
 Sort  (cost=2439.46..2455.45 rows=6394 width=48) (actual time=14.223..14.709 rows=6262 loops=1)
   Sort Key: release_date
   Sort Method: quicksort  Memory: 738kB
   ->  Seq Scan on album  (cost=0.00..2035.28 rows=6394 width=48) (actual time=0.008..7.861 rows=6262 loops=1)
         Filter: (((release_date)::text >= '2000'::text) AND ((release_date)::text <= '2010'::text))
         Rows Removed by Filter: 80890
 Planning Time: 0.091 ms
 Execution Time: 14.878 ms
(8 rows)
```

Q8 Albums by release period — After indexing — Execution Time: 8.327 ms

Execution plan after the six proposed indexes were created and ANALYZE ran. Plan evidence to show: idx_album_release_date.



```
Time: 0.736 ms
INDEXED Q8 - albums released during a period
                                                                 QUERY PLAN
--------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=1327.84..1343.87 rows=6414 width=48) (actual time=7.983..8.176 rows=6262 loops=1)
   Sort Key: release_date
   Sort Method: quicksort  Memory: 738kB
   ->  Bitmap Heap Scan on album  (cost=98.04..922.25 rows=6414 width=48) (actual time=0.953..2.594 rows=6262 loops=1)
         Recheck Cond: (((release_date)::text >= '2000'::text) AND ((release_date)::text <= '2010'::text))
         Heap Blocks: exact=698
         ->  Bitmap Index Scan on idx_album_release_date  (cost=0.00..96.43 rows=6414 width=0) (actual time=0.895..0.896 rows=6262 loops=1)
               Index Cond: (((release_date)::text >= '2000'::text) AND ((release_date)::text <= '2010'::text))
 Planning Time: 0.179 ms
 Execution Time: 8.327 ms
(10 rows)
```

Source: index_benchmark_output.txt. Include the original benchmark file with the submission so all plan lines can be checked.